

WHO ARE WE?

We are from Argentina, participating for the second time in the international RoboCup competition, in the Rescue Simulation category. Previously, we took part in RoboCup Americas 2025 in which we got the first place and then in RoboCup Salvador 2025 in which we got third place. This poster presents the most important aspects of our robot's software, highlighting the strategies and improvements developed for RoboCup 2026.

TEAM:



Martina Talamona
Programming & testing.



Ramiro Francavilla
Programming & testing.



Emanuel Hamul
Mentor.

ROBOT'S HARDWARE COMPONENTS

- Three 40x40 **cameras** (left, front and right) for robust visual analysis, enabling the detection of signs, fake tokens and Cognitives targets.
- A differential drive system with two independently controlled **wheels** for precise movement and smooth turns.
- A **distance sensor** for obstacles detection aided by Lidar.
- A **LIDAR** sensor provides high-resolution 2D mapping and accurate obstacle detection, improving path planning when combined with camera system.
- GPS**: due to its noise the Gps is still being fundamental, the robot averages multiple readings while stationary and applies stronger corrections after a LOP event to maintain a stable position estimate.
- Inertial unit**: For directional alignment.

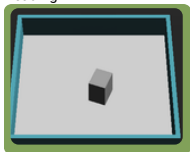
ROBOT'S SOFTWARE

- We use Python as our programming language. Our code follows a modular structure with over 10 specialized modules for tasks such as navigation, mapping, image processing, Vector for mathematical functions, tile classification, and robot control.
- This design simplifies maintenance, testing, and expansion by assigning clear responsibilities to each subsystem and supporting efficient team collaboration.

INNOVATIONS

- We split obstacle detection using Lidar and the front camera; Lidar to see if there is something close, and as a helper is the front camera, we take the left most, the right most and the center pixel and we compare those values excluding walls and token colours. Once the obstacle is confirmed, they are marked as "X" in the matrix representation.
- We also implemented a swamps avoidance navigation, where each detected swamps pixel receives an additional cost, and it's minitile visit count is increased proportionally to further discourage re-entry.
- We implemented a positioning based in the LiDAR's front ray (ray 256), each time the robot moves, the difference between the previous and current front-ray distance is used to calculate how far it traveled in X and Y based on its heading.

EXAMPLE OF HOW WE REPRESENT OBSTACLES:



CAETI-UAI



We fail. We learn. We improve. We try again.
That is our principle.

TECHNOLOGIES



WALL TOKEN DETECTION

Uses a custom image processing method to detect victims and cognitive targets on walls, using the front and left cameras. A color threshold highlights key colors and removes background noise, then contours are extracted to identify potential signs. Victims are recognized by analyzing black and white pixel distribution across five regions in a 3x3 grid. Cognitive targets are confirmed through shape and color validation – near-square contour, minimum area, valid color present – with no letter analysis needed.



NAVIGATION SYSTEM

We use a hybrid pathfinding system combining Dijkstra's algorithm for global planning and Greedy Best First Search for precise, pixel-level navigation in tight spaces. The map grows dynamically with LiDAR data, adapting to any environment size. Obstacles like walls and black holes are marked as non-traversable. Swamp tiles are assigned an additional traversal cost, discouraging re-entry while still allowing passage if no other route exists. Straight paths are prioritized to optimize movement, straight paths are prioritized, and the final route is smoothed to eliminate unnecessary steps. We use Godot Engine to create a real-time visualization. This allowed us to improve our navigation algorithm.

VISUALIZER (GODOT ENGINE)

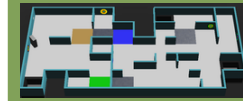
- In the visualizer we can see the cost of each minitile(g) and the number of walls(w) it has around it.
- Points painted as violet are the lowest cost path that the robot has planned.
- Black color represent walls and obstacles.



MAPPING

Our mapping system constructs a dynamic, grid-based map as the robot explores. Each tile is subdivided into four sections to allow precise classification of terrain types and obstacles. LiDAR sensors detect walls and continuously update the map in real time. As the robot moves, the map expands automatically to include newly discovered areas. Tiles are classified using sensor input, distinguishing features such as swamp, checkpoint, or black hole. Obstacles are marked as non-traversable and recorded as "X" in the matrix representation, while visited tiles are displayed in green. For efficient pathfinding and local analysis, the system maintains a cropped matrix centered around the robot's current position.

MAP



MATRIX WITH THE MAP:

